Foto AI Agent - Complete Implementation Specification

Executive Summary

Transform the current Foto photography coaching chatbot into a **proactive, autonomous AI Agent** that intelligently coaches the user based on their real-world schedule, photography patterns, location, and weather conditions. The agent works 24/7 in the background to generate personalized challenges, send timely notifications, track progress, and adapt to user behavior.

Table of Contents

1. Core Concept & Transformation
2. User Context & Schedule
3. Technical Architecture
4. Database Schema
5. External Integrations
6. AI Agent Intelligence Logic
7. Challenge Type System
8. Notification Strategy
9. User Experience Flows
10. Dashboard & UI Components
11. Scheduled Tasks & Automation
12. Progress Tracking & Analytics
13. API Endpoints
14. Implementation Steps
15. Testing Strategy
16. Deployment Considerations

---

1. Core Concept & Transformation

Current State

- **Reactive chatbot**: User visits website → chats with AI → gets responses
- User initiates all interactions
- No proactive coaching
- No progress tracking
- No context awareness

AI Agent Version

- **Proactive autonomous system**: Monitors user's schedule, location, weather

- Sends daily challenges automatically
- Adapts to user behavior and completion patterns
- Tracks progress and skill development
- Works 24/7 in the background
- Context-aware: knows work schedule, photography availability, location

---

2. User Context & Schedule

User Profile: Seun's Photography Lifestyle

```
{
  // Core Schedule
  workSchedule: {
    mondayToFriday: {
      workHours: "9am-5pm",
      lunchBreak: "12pm-1pm",
      afterWork: "5pm-7pm"
    },
    workFromHomeDays: "varies" // Auto-detected from calendar
  },

  // Photography Availability
  primaryPhotographyDay: "Saturday", // Main shooting day (4-8 hours)
  secondaryDays: ["Sunday"], // Sometimes
  lunchBreakPhotography: true, // 20-30 mins quick shots
  eveningPhotography: true, // 1-2 hours golden hour
  wfhFlexibility: true, // Flexible throughout day when WFH

  // Locations
  locations: {
    weekend: "Explore new areas",
    lunchBreak: "Near office",
    afterWork: "Commute route + neighborhood",
    workFromHome: "Home + local neighborhood"
  },

  // Gear
  gear: [
    { camera: "Fujifilm X-T5", type: "primary", use: "weekend deep dives" },
    { camera: "Ricoh GR IIIx", type: "everyday", use: "lunch breaks, WFH" },
    { camera: "Nikon FE2", type: "film", use: "special projects" },
    { camera: "Pentax ME", type: "film", use: "special projects" }
  ],

  // Skill & Interests
  skillLevel: "intermediate",
  interests: ["street", "portrait", "architecture"],
  avoids: ["macro", "sports"]
}
```

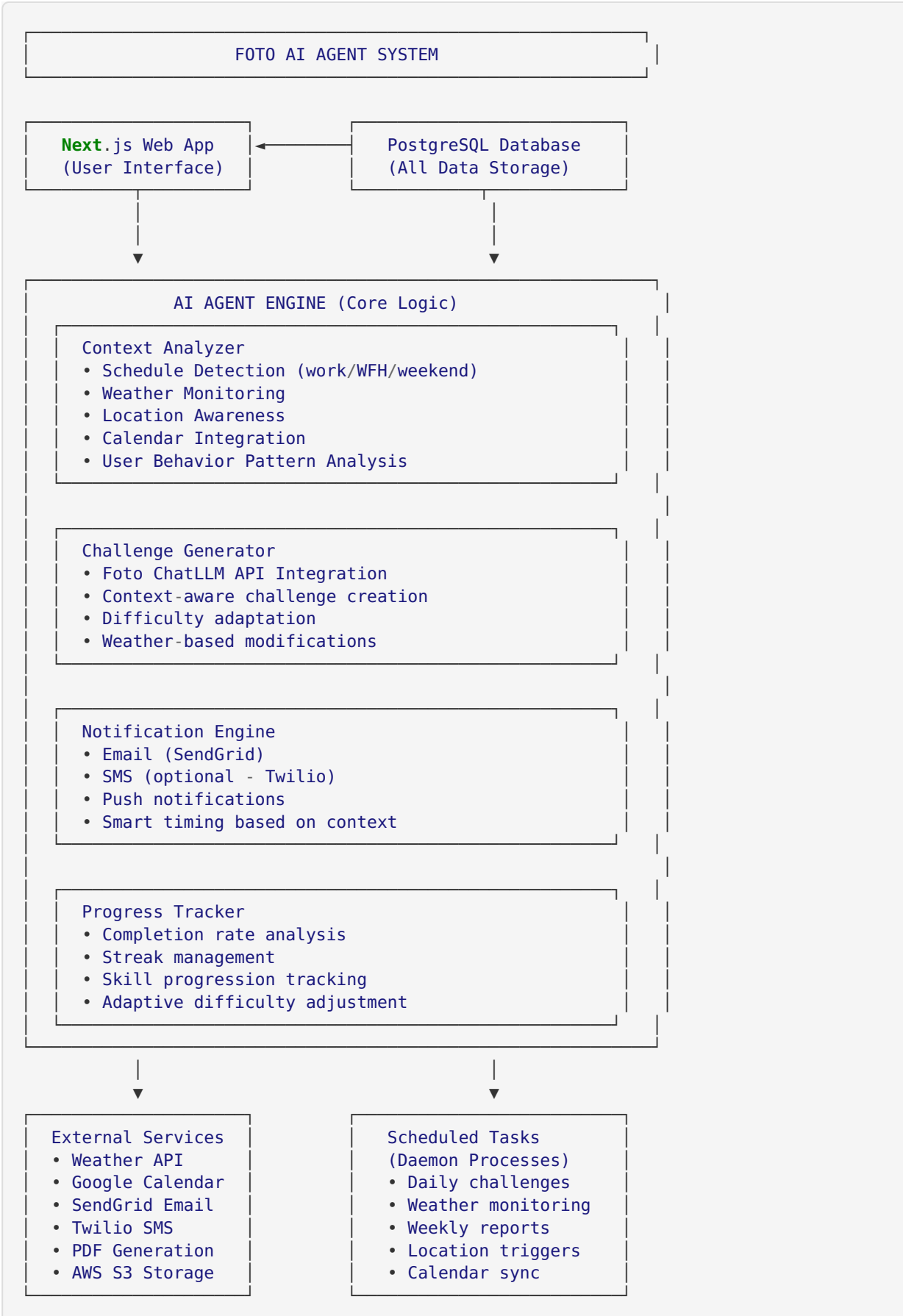
Key Insight: Context-Aware Coaching

The AI Agent must understand that:
- **Saturday** is sacred photography time → Send complex, detailed challenges
- **Weekdays** are busy with work → Send only quick, achievable micro-challenges
- **Lunch breaks** are limited (15-25 min) → Single-shot focus

- **Work-from-home days** offer flexibility → Adaptable timing
- **Evening golden hour** opportunities → Weather-dependent alerts
- **Weather** impacts feasibility → Adapt or skip challenges

---

3. Technical Architecture



4. Database Schema

Complete Prisma Schema

```
// Foto AI Agent - Database Schema

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ============================================
// USER PROFILE & AUTHENTICATION
// ============================================

model User {
  id        String   @id @default(cuid())
  email     String   @unique
  password  String
  name      String
  timezone  String   @default("America/Los_Angeles")
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  // Relations
  preferences       UserPreferences?
  gear              UserGear[]
  challenges        DailyChallenge[]
  itineraries       Itinerary[]
  progress          UserProgress?
  agentActions      AgentAction[]
  submissions       ChallengeSubmission[]
}

// ============================================
// USER PREFERENCES & SCHEDULE
// ============================================

model UserPreferences {
  id     String @id @default(cuid())
  userId String @unique
  user   User   @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Work Schedule
  workDays          String[]  @default(["Mon", "Tue", "Wed", "Thu", "Fri"])
  workStartTime     String    @default("09:00")
  workEndTime       String    @default("17:00")
  lunchStartTime    String    @default("12:00")
  lunchEndTime      String    @default("13:00")

  // Photography Availability
  primaryPhotoDay        String    @default("Saturday")
  secondaryPhotoDays     String[]  @default(["Sunday"])
  lunchBreakPhotography  Boolean   @default(true)
  eveningPhotography     Boolean   @default(true)
  wfhFlexibility         Boolean   @default(true)

  // Skill & Interests
  skillLevel              String    @default("intermediate")
  photographyInterests    String[]  @default(["street", "portrait", "architecture"])
  avoidsStyles            String[]  @default([])
```

```
  // Challenge Preferences
  saturdayComplexity    String    @default("high")
  weekdayComplexity     String    @default("micro")
  adaptToWeather        Boolean   @default(true)
  adaptToCalendar       Boolean   @default(true)

  // Notification Preferences
  saturdayNotifTime     String    @default("07:00")
  saturdayNotifChannels String[]  @default(["email"])
  weekdayNotifTime      String    @default("11:30")
  weekdayNotifChannels  String[]  @default(["email"])
  eveningAlertsEnabled  Boolean   @default(true)
  wfhNotifTime          String    @default("09:00")

  // Location
  officeLocation    String?
  homeLocation      String?
  currentLocation   String?

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

// ============================================
// USER GEAR INVENTORY
// ============================================

model UserGear {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete:
Cascade)

  category    String   // camera, lens, film, accessory
  brand       String
  model       String
  specs       String?  // JSON string with specifications
  isPrimary   Boolean  @default(false)

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

// ============================================
// DAILY CHALLENGES
// ============================================

model DailyChallenge {
  id              String   @id @default(cuid())
  userId          String
  user            User     @relation(fields: [userId], references: [id], onDelete: Cas
cade)

  challengeType   String   // WEEKEND_DEEP_DIVE, LUNCH_MICRO, AFTER_WORK, WORK_FROM_HO
ME
  challengeText   String   @db.Text
  difficulty      String   // micro, short, medium, deep
  category        String   // street, portrait, architecture, etc.
  duration        String   // "15-25 mins", "2-4 hours"

  // Context
  weatherConditions  String?
  location           String?
```

```
  recommendedGear    String[]  @default([])
  tips               String?   @db.Text

  // Scheduling
  issuedDate      DateTime
  scheduledTime   String?
  bestTimeStart   String?
  bestTimeEnd     String?

  // Status
  status          String   @default("pending") // pending, accepted, skipped, com-
pleted
  completedDate   DateTime?

  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  // Relations
  submissions     ChallengeSubmission[]
}

// ============================================
// CHALLENGE SUBMISSIONS
// ============================================

model ChallengeSubmission {
  id            String   @id @default(cuid())
  challengeId   String
  challenge     DailyChallenge @relation(fields: [challengeId], references: [id], onDe
lete: Cascade)
  userId        String
  user          User     @relation(fields: [userId], references: [id], onDelete: Cas-
cade)

  photoUrl      String?
  notes         String?  @db.Text
  feedback      String?  @db.Text  // AI-generated feedback
  rating        Int?     // User self-rating 1-5

  submittedAt   DateTime @default(now())
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
}

// ============================================
// LOCATION-BASED ITINERARIES
// ============================================

model Itinerary {
  id                String   @id @default(cuid())
  userId            String
  user              User     @relation(fields: [userId], references: [id], onDelete: C
ascade)

  location          String
  title             String
  description       String   @db.Text

  // Details
  spots             String   @db.Text  // JSON array of spot details
  suggestedTimes    String[] @default([])
  weatherConditions String?
  gearRecommendations String[] @default([])
```

```
  duration          String?

  // PDF & Calendar
  pdfUrl            String?
  calendarEventId   String?

  // Status
  status            String   @default("generated")
  generatedDate     DateTime @default(now())
  scheduledDate     DateTime?
  completedDate     DateTime?

  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt
}

// ============================================
// USER PROGRESS TRACKING
// ============================================

model UserProgress {
  id        String @id @default(cuid())
  userId    String @unique
  user      User   @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Stats
  totalChallengesIssued    Int @default(0)
  totalChallengesCompleted Int @default(0)
  currentStreak            Int @default(0)
  longestStreak            Int @default(0)

  // Completion Rates by Type
  saturdayChallengesCompleted  Int @default(0)
  saturdayChallengesIssued     Int @default(0)
  lunchBreakCompleted          Int @default(0)
  lunchBreakIssued             Int @default(0)
  afterWorkCompleted           Int @default(0)
  afterWorkIssued              Int @default(0)
  wfhCompleted                 Int @default(0)
  wfhIssued                    Int @default(0)

  // Learning & Adaptation
  favoriteSubjects      String[] @default([])
  improvementAreas      String[] @default([])
  skillGrowthMetrics    String?  @db.Text  // JSON

  // Last Activity
  lastChallengeDate     DateTime?
  lastCompletionDate    DateTime?
  lastStreakUpdate      DateTime?

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

// ============================================
// AGENT ACTIONS LOG
// ============================================

model AgentAction {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete:
```

```
Cascade)

  actionType  String   // CHALLENGE_SENT, WEATHER_ALERT, ITINERARY_GENERATED, WEEKLY_R
EPORT
  details     String   @db.Text  // JSON with action details
  success     Boolean  @default(true)
  errorMessage String?

  executedAt  DateTime @default(now())
  createdAt   DateTime @default(now())
}
```

---

5. External Integrations

Required Services

1. Weather API - OpenWeatherMap

- **Purpose**: Real-time weather monitoring for challenge adaptation
- **Endpoint**: `https://api.openweathermap.org/data/2.5/forecast`
- **Free Tier**: 60 calls/minute
- **Required**: API_KEY
- **Usage**:
  ``javascript
  // Get 5-day forecast
  const response = await fetch(https://api.openweathermap.org/data/2.5/forecast?lat=${lat}
  &lon=${lon}&appid=${API_KEY}&units=imperial`
  );

// Check conditions
const data = await response.json();
const today = data.list[0];
const condition = today.weather[0].main; // “Clear”, “Rain”, “Fog”, etc.
const temp = today.main.temp;
```

### 2. Email Service - SendGrid

- **Purpose**: Send daily challenges, alerts, weekly reports
- **Endpoint**: `https://api.sendgrid.com/v3/mail/send`
- **Free Tier**: 100 emails/day
- **Required**: API_KEY
- **Usage**:
  ```javascript
  const sgMail = require(‘@sendgrid/mail’);
  sgMail.setApiKey(process.env.SENDGRID_API_KEY);

const msg = {
to: user.email,
from: ‘foto@yourdomain.com’,
subject: ‘Your Foto Challenge: Golden Hour Portraits’,
html: emailTemplate,
};
```
```

```
await sgMail.send(msg);
```

3. SMS Service - Twilio (Optional)

- **Purpose**: Quick alerts for golden hour, weather opportunities
- **Cost**: ~$0.0075 per message
- **Required**: ACCOUNT_SID, AUTH_TOKEN, PHONE_NUMBER
- **Usage**:
  ```javascript
  const twilio = require('twilio');
  const client = twilio(ACCOUNT_SID, AUTH_TOKEN);

await client.messages.create({
body: '🌅 Golden hour alert! Perfect light in 30 mins.',
from: TWILIO_PHONE,
to: user.phone
});
```

4. Google Calendar API

- **Purpose**: Detect WFH days, add photo walk events
- **Required**: OAuth 2.0 setup
- **Usage**:
  ```javascript
  // Check for WFH events
  const events = await calendar.events.list({
  calendarId: 'primary',
  timeMin: startOfDay,
  timeMax: endOfDay,
  q: 'work from home'
  });

const isWFH = events.data.items.length > 0;
```

5. PDF Generation - jsPDF or Puppeteer

- **Purpose**: Generate printable itineraries, reports
- **Usage**:
  ```javascript
  const jsPDF = require('jspdf');
  const doc = new jsPDF();

doc.text('Weekend Photography Itinerary', 10, 10);
doc.text(itineraryContent, 10, 20);
doc.save('itinerary.pdf');
```

6. AWS S3 - Cloud Storage

- **Purpose**: Store user photo submissions, PDFs
- **Already configured**: Via initialize_cloud_storage
- **Usage**: Store files with cloud_storage_path in database

7. Foto ChatLLM API

- **Purpose**: Generate personalized challenges, feedback, itineraries
- **Already configured**: Via ABACUSAI_API_KEY
- **Model**: Use the existing Foto ChatLLM (App ID: 13a86b8346)
- **Usage**:

```
javascript
 const response = await fetch('https://apps.abacus.ai/v1/chat/completions', {
   method: 'POST',
   headers: {
     'Content-Type': 'application/json',
     'Authorization': `Bearer ${process.env.ABACUSAI_API_KEY}`
   },
   body: JSON.stringify({
     model: 'gpt-4.1-mini',
     messages: [{
       role: 'system',
       content: 'You are Foto, an expert photography coach...'
     }, {
       role: 'user',
       content: `Generate a ${challengeType} challenge for ${context}`
     }],
     stream: false,
     max_tokens: 1500
   })
 });
```

---

6. AI Agent Intelligence Logic

Core Decision Engine

```
/**
 * AI Agent Decision Logic - Challenge Determination
 * This is the brain of the AI Agent
 */

async function determineAndSendChallenge(userId) {
  // 1. GET CONTEXT
  const user = await db.user.findUnique({
    where: { id: userId },
    include: { preferences: true, progress: true, gear: true }
  });

  const now = new Date();
  const dayOfWeek = now.getDay(); // 0=Sun, 6=Sat
  const hour = now.getHours();

  // 2. CHECK CALENDAR FOR WFH
  const isWorkFromHome = await checkGoogleCalendar(user, now);

  // 3. GET WEATHER
  const weather = await getWeatherForecast(user.preferences.homeLocation);

  // 4. GET USER HISTORY
  const recentChallenges = await db.dailyChallenge.findMany({
    where: {
      userId: userId,
      issuedDate: { gte: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) }
    },
    orderBy: { issuedDate: 'desc' }
  });

  const completionRate = calculateCompletionRate(recentChallenges);

  // 5. APPLY DECISION LOGIC
  let challengeParams = {};

  // ========================================
  // SATURDAY - MAIN PHOTOGRAPHY DAY
  // ========================================
  if (dayOfWeek === 6) {
    challengeParams = {
      type: "WEEKEND_DEEP_DIVE",
      sendTime: "07:00",
      difficulty: "deep",
      duration: "4-8 hours",
      location: "new area exploration",
      gear: recommendFullKit(user.gear),
      complexity: "high",
      includeItinerary: true,
      includePDF: true,
      addToCalendar: true
    };

    // Weather adaptation
    if (weather.condition === "Rain" || weather.condition === "Thunderstorm") {
      challengeParams.title = "Rainy Day Photography Mastery";
      challengeParams.options = [
        { type: "indoor", location: "museum or market", gear: "any" },
        { type: "outdoor", location: "embrace rain", gear: "X-T5 (weather-sealed)" },
        { type: "postpone", message: "Move to Sunday if weather improves" }
      ];
    }
```

```
  }

  // ========================================
  // SUNDAY - LIGHTER WEEKEND
  // ========================================
  else if (dayOfWeek === 0) {
    challengeParams = {
      type: "WEEKEND_LITE",
      sendTime: "08:00",
      difficulty: "medium",
      duration: "2-3 hours",
      optional: true,
      relaxed: true,
      location: "neighborhood or familiar area",
      gear: recommendCompactKit(user.gear)
    };
  }

  // ========================================
  // WEEKDAYS - WORK DAYS
  // ========================================
  else if (dayOfWeek >= 1 && dayOfWeek <= 5) {

    // WORK FROM HOME
    if (isWorkFromHome) {
      challengeParams = {
        type: "WORK_FROM_HOME",
        sendTime: "09:00",
        difficulty: "short",
        duration: "30 mins flexible",
        location: "home + neighborhood",
        gear: "Ricoh GR IIIx (grab and go)",
        flexible: true,
        reminderTime: "15:00" // Afternoon reminder
      };

      // Special weather opportunities
      if (weather.condition === "Fog" || weather.condition === "Mist") {
        challengeParams.alert = {
          type: "WEATHER_OPPORTUNITY",
          message: "🌫️ Rare fog! Perfect for moody neighborhood shots",
          urgent: true
        };
      }
    }

    // REGULAR OFFICE DAY
    else {
      challengeParams = {
        type: "LUNCH_MICRO",
        sendTime: "11:30", // Just before lunch
        difficulty: "micro",
        duration: "15-25 mins",
        location: "near office",
        gear: "Ricoh GR IIIx (pocket-friendly)",
        singleShot: true,
        simple: true
      };

      // Check for evening golden hour opportunity
      if (user.preferences.eveningPhotography) {
        const sunsetTime = weather.sunset;
        const goldenHourStart = new Date(sunsetTime);
```

```
        goldenHourStart.setHours(goldenHourStart.getHours() - 1);

        if (weather.cloudCover < 70 && weather.visibility > 8) {
          challengeParams.eveningBonus = {
            type: "GOLDEN_HOUR_ALERT",
            sendTime: "16:30",
            duration: "45-90 mins",
            location: "commute or neighborhood",
            gear: "X-T5 or Ricoh",
            message: "🌅 Beautiful sunset predicted! Golden hour photography opportun-
ity"
          };
        }
      }

      // Skip if bad weather
      if (weather.condition === "Heavy Rain" || weather.condition === "Thunderstorm")
{
        challengeParams = null; // Skip lunch challenge
      }
    }
  }

  // ========================================
  // ADAPTIVE DIFFICULTY ADJUSTMENT
  // ========================================

  // If user completion rate is low, simplify
  if (completionRate.overall < 0.4) {
    if (challengeParams.difficulty === "medium") {
      challengeParams.difficulty = "short";
    } else if (challengeParams.difficulty === "short") {
      challengeParams.difficulty = "micro";
    }
  }

  // If user completion rate is high, can increase challenge
  if (completionRate.overall > 0.8 && challengeParams.difficulty === "micro") {
    challengeParams.difficulty = "short";
  }

  // ========================================
  // GENERATE CHALLENGE WITH FOTO CHATLLM
  // ========================================

  if (!challengeParams) {
    // No challenge today
    await logAgentAction(userId, "NO_CHALLENGE", "Skipped due to conditions");
    return;
  }

  const challenge = await generateChallengeWithAI(user, challengeParams, weather);

  // ========================================
  // SAVE TO DATABASE
  // ========================================

  const savedChallenge = await db.dailyChallenge.create({
    data: {
      userId: userId,
      challengeType: challengeParams.type,
      challengeText: challenge.text,
      difficulty: challengeParams.difficulty,
```

```
      category: challenge.category,
      duration: challengeParams.duration,
      weatherConditions: weather.description,
      location: challengeParams.location,
      recommendedGear: challengeParams.gear,
      tips: challenge.tips,
      issuedDate: now,
      scheduledTime: challengeParams.sendTime,
      bestTimeStart: challenge.bestTimeStart,
      bestTimeEnd: challenge.bestTimeEnd
    }
  });

  // ========================================
  // UPDATE PROGRESS TRACKING
  // ========================================

  await updateProgressStats(userId, {
    type: challengeParams.type,
    action: "issued"
  });

  // ========================================
  // SEND NOTIFICATION
  // ========================================

  await sendNotification(user, savedChallenge, challengeParams);

  // ========================================
  // LOG ACTION
  // ========================================

  await logAgentAction(userId, "CHALLENGE_SENT", {
    challengeId: savedChallenge.id,
    type: challengeParams.type,
    weather: weather.condition,
    isWFH: isWorkFromHome
  });

  return savedChallenge;
}

/**
 * Generate challenge content using Foto ChatLLM API
 */
async function generateChallengeWithAI(user, params, weather) {
  const prompt = `Generate a ${params.difficulty} photography challenge for ${params.t
ype}.

User Profile:
- Name: ${user.name}
- Skill Level: ${user.preferences.skillLevel}
- Interests: ${user.preferences.photographyInterests.join(', ')}
- Available Time: ${params.duration}
- Location: ${params.location}
- Gear: ${params.gear.join(', ')}

Context:
- Weather: ${weather.condition}, ${weather.temp}°F
- Day: ${params.type === 'WEEKEND_DEEP_DIVE' ? 'Saturday (main photography day)' : 'We
ekday'}
${params.flexible ? '- User has flexible timing today (work from home)' : ''}
```

```
Requirements:
- Duration: ${params.duration}
- Difficulty: ${params.difficulty}
${params.singleShot ? '- Focus: Single compelling shot' : '- Focus: Series or pro-
ject'}

Generate a challenge that is:
1. Achievable in the given time
2. Appropriate for the weather
3. Matches user's skill level and interests
4. Specific and actionable
5. Inspiring and educational

Format:
{
  "title": "Challenge title",
  "text": "Detailed challenge description with specific goals",
  "category": "street/portrait/architecture",
  "tips": "3-5 specific tips to help succeed",
  "bestTimeStart": "HH:MM",
  "bestTimeEnd": "HH:MM"
}`;

  const response = await fetch('https://apps.abacus.ai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.ABACUSAI_API_KEY}`
    },
    body: JSON.stringify({
      model: 'gpt-4.1-mini',
      messages: [
        {
          role: 'system',
          content: 'You are Foto, an expert photography coach. Generate personalized,
achievable photography challenges.'
        },
        {
          role: 'user',
          content: prompt
        }
      ],
      response_format: { type: "json_object" },
      max_tokens: 1500
    })
  });

  const data = await response.json();
  return JSON.parse(data.choices[0].message.content);
}
```

7. Challenge Type System

Challenge Types & Specifications

A. WEEKEND_DEEP_DIVE (Saturday)

```
{
  duration: "4-8 hours",
  complexity: "high",
  planning: "required",

  components: [
    "Full itinerary with multiple locations",
    "Shot list (10-20 photos)",
    "Specific techniques to practice",
    "Time-based shooting plan",
    "Gear recommendations (full kit)"
  ],

  deliverables: [
    "Email with full challenge description",
    "PDF itinerary (downloadable)",
    "Google Calendar event",
    "Weather-adapted plan",
    "Map with photo spots"
  ],

  examples: [
    "Urban Geometry: Architecture + street life in new neighborhood",
    "Golden Hour Portrait Series: 10 portraits in 3 lighting setups",
    "Film Photography Day: Shoot entire roll on Nikon FE2",
    "Street Photography Project: Document market or event"
  ]
}
```

B. WEEKEND_LITE (Sunday)

```
{
  duration: "2-3 hours",
  complexity: "medium",
  planning: "light",
  optional: true,

  components: [
    "Relaxed single-location shoot",
    "Theme-based exploration",
    "No pressure, enjoy the process",
    "Familiar location acceptable"
  ],

  examples: [
    "Neighborhood Walk: Find 10 interesting moments",
    "Coffee Shop Photography: Natural light + people",
    "Park Exploration: Nature + street life blend",
    "Lazy Sunday Portraits: Friends/family casual shots"
  ]
}
```

C. LUNCH_MICRO (Weekday Lunch)

```
{
  duration: "15-25 mins",
  complexity: "micro",
  focus: "single shot",
  location: "near office",

  components: [
    "One compelling photograph",
    "Simple, achievable goal",
    "Pocket camera (Ricoh GR IIIx)",
    "No planning needed"
  ],

  examples: [
    "Catch-the-Light: One interesting shadow pattern",
    "Street Detail: One compelling texture or pattern",
    "Human Moment: One candid expression",
    "Geometry: One architectural detail",
    "Quick Portrait: One environmental portrait"
  ]
}
```

D. AFTER_WORK (Weekday Evening)

```
{
  duration: "45-90 mins",
  complexity: "short",
  trigger: "golden hour opportunity",
  location: "commute route or neighborhood",

  components: [
    "Golden hour focused",
    "5-10 photos",
    "Can be done on the way home",
    "Weather dependent"
  ],

  examples: [
    "Commute Stories: Document journey in 5 shots",
    "Golden Hour Glow: Sunset portraits",
    "Evening Rush: Street life in golden light",
    "Neighborhood Magic: Explore local area"
  ]
}
```

E. WORK_FROM_HOME (Flexible)

```
{
  duration: "30-60 mins",
  complexity: "short",
  flexibility: "high",
  location: "home + local neighborhood",

  components: [
    "Can be done anytime during day",
    "Reminder sent in afternoon",
    "Home or local area",
    "Relaxed, no rush"
  ],

  examples: [
    "Domestic Details: Beauty in everyday objects",
    "Window Light: Indoor portraits",
    "Neighborhood Walk: 30-min local exploration",
    "Home Studio: Still life with natural light"
  ]
}
```

Challenge Intensity Matrix

```
TIME AVAILABLE vs CHALLENGE COMPLEXITY

              Micro       Short       Medium      Deep
              (15min)     (30min)     (1-2hr)     (4-8hr)

Lunch         ████        ▓▓▓▓        ▒▒▒▒        ▒▒▒▒
Break         IDEAL       Tight       No          No

After         ▓▓▓▒        ████        ▓▓▓▒        ▒▒▒▒
Work          OK          IDEAL       Tight       No

WFH           ▓▓▓▓        ████        ████        ▓▓▓▒
Day           Good        Good        IDEAL       Maybe

Weekend       ▓▓▓▓        ▓▓▓▓        ████        ████
              Warmup      Warmup      Good        IDEAL
```

8. Notification Strategy

Notification Timing & Channels

```
const NOTIFICATION_STRATEGY = {

  // SATURDAY - Main Photography Day
  saturday: {
    primaryTime: "07:00",
    channels: ["email"],
    content: {
      subject: "🎯 Weekend Photography Challenge + Full Itinerary",
      includes: [
        "Detailed challenge description",
        "PDF itinerary attachment",
        "Weather forecast",
        "Gear checklist",
        "Location map",
        "Calendar event link"
      ]
    },
    followUp: {
      time: "18:00",
      message: "How did it go? Share your favorite shot!",
      channels: ["email"]
    }
  },

  // SUNDAY - Optional
  sunday: {
    primaryTime: "08:00",
    channels: ["email"],
    content: {
      subject: "☀️ Optional Sunday Challenge (No Pressure!)",
      tone: "relaxed",
      includes: ["Light challenge", "Optional"]
    }
  },

  // WEEKDAY LUNCH
  weekday: {
    primaryTime: "11:30", // 30 min before lunch
    channels: ["email"],
    content: {
      subject: "☕ Quick Lunch Break Challenge (15 mins)",
      tone: "quick and simple",
      includes: ["Single shot focus", "Near office"]
    },
    skip: {
      conditions: ["heavy_rain", "thunderstorm", "extreme_cold"]
    }
  },

  // WORK FROM HOME
  workFromHome: {
    primaryTime: "09:00",
    channels: ["email"],
    content: {
      subject: "🏠 WFH Flexible Challenge (Whenever Convenient)",
      tone: "flexible and relaxed",
      includes: ["Do anytime today", "Home + neighborhood"]
    },
    reminder: {
      time: "15:00",
      message: "Reminder: Take 20 mins for your photo challenge?"
    }
```

```
  },

  // EVENING GOLDEN HOUR
  eveningAlert: {
    primaryTime: "16:30", // 90 min before sunset
    channels: ["email"],
    trigger: {
      conditions: ["clear_sky", "partly_cloudy"],
      userPreference: "eveningPhotography === true"
    },
    content: {
      subject: "🌅 Golden Hour Alert! Perfect Light Tonight",
      urgent: true,
      includes: ["Sunset time", "Route suggestion", "Duration"]
    }
  },

  // WEATHER OPPORTUNITIES
  weatherAlert: {
    channels: ["email"],
    triggers: [
      { condition: "fog", message: "🌫️ Rare fog! Moody photography opportunity" },
      { condition: "snow", message: "❄️ Fresh snow! Winter photography time" },
      { condition: "clearing_storm", message: "⛈️ Storm clearing! Dramatic skies
ahead" }
    ]
  },

  // WEEKLY REPORT
  weeklyReport: {
    dayOfWeek: "Sunday",
    time: "20:00",
    channels: ["email"],
    content: {
      subject: "📊 Your Week in Photography",
      includes: [
        "Challenges completed",
        "Completion rate",
        "Skill improvements",
        "Next week focus",
        "PDF report attachment"
      ]
    }
  }
};
```

Email Templates

Saturday Challenge Email

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; }
    .header { background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
              color: white; padding: 30px; text-align: center; }
    .content { padding: 30px; background: #f9fafb; }
    .challenge-box { background: white; padding: 20px; border-radius: 8px;
                     margin: 20px 0; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
    .button { background: #667eea; color: white; padding: 12px 24px;
              text-decoration: none; border-radius: 6px; display: inline-block; }
  </style>
</head>
<body>
  <div class="header">
    <h1>🎯 Weekend Photography Challenge</h1>
    <p>Saturday, [DATE]</p>
  </div>

  <div class="content">
    <div class="challenge-box">
      <h2>[CHALLENGE TITLE]</h2>
      <p><strong>Duration:</strong> 4-8 hours</p>
      <p><strong>Location:</strong> [LOCATION]</p>
      <p><strong>Weather:</strong> [WEATHER FORECAST]</p>

      <h3>Your Challenge:</h3>
      <p>[CHALLENGE DESCRIPTION]</p>

      <h3>Recommended Gear:</h3>
      <ul>
        <li>Fujifilm X-T5</li>
        <li>23mm and 50mm lenses</li>
        <li>Extra batteries</li>
      </ul>

      <h3>Tips for Success:</h3>
      <ol>
        <li>[TIP 1]</li>
        <li>[TIP 2]</li>
        <li>[TIP 3]</li>
      </ol>

      <h3>Best Times to Shoot:</h3>
      <p>🌅 Morning: 9:00 AM - 12:00 PM (soft light)</p>
      <p>🌆 Evening: 4:00 PM - 7:00 PM (golden hour)</p>
    </div>

    <p style="text-align: center;">
      <a href="[ITINERARY_PDF_LINK]" class="button">Download Full Itinerary (PDF)</a>
      <a href="[CALENDAR_LINK]" class="button">Add to Calendar</a>
    </p>

    <p style="text-align: center; margin-top: 30px;">
      <a href="[APP_URL]/challenges/[ID]">View in Dashboard</a> |
      <a href="[APP_URL]/challenges/[ID]/skip">Skip This Challenge</a>
    </p>
  </div>
</body>
</html>
```

Lunch Break Micro Challenge Email

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; }
    .header { background: #10b981; color: white; padding: 20px; text-align: center; }
    .content { padding: 20px; }
    .quick-tip { background: #fef3c7; padding: 15px; border-left: 4px solid #f59e0b;
                 margin: 15px 0; border-radius: 4px; }
  </style>
</head>
<body>
  <div class="header">
    <h2>☕ Quick Lunch Break Challenge</h2>
    <p>15-25 minutes | One compelling shot</p>
  </div>

  <div class="content">
    <h3>[CHALLENGE TITLE]</h3>
    <p>[CHALLENGE DESCRIPTION]</p>

    <div class="quick-tip">
      <strong>💡 Quick Tip:</strong> [TIP]
    </div>

    <p><strong>📷 Gear:</strong> Ricoh GR IIIx (pocket-friendly)</p>
    <p><strong>📍 Location:</strong> Near your office</p>
    <p><strong>⏱️ Time:</strong> 15-25 minutes</p>

    <p style="margin-top: 30px; text-align: center; font-size: 14px; color: #6b7280;">
      Too busy today? <a href="[APP_URL]/challenges/[ID]/skip">Skip this challenge</a>
    </p>
  </div>
</body>
</html>
```

9. User Experience Flows

Typical Week in the Life of a Foto User

```
MONDAY (Office Day)
├─ 11:30 AM: Email - "Lunch Break Micro-Challenge: Shadow Patterns"
│  └─ User: Checks email, goes out for lunch, snaps one shadow photo with Ricoh
├─ 4:45 PM: Email - "🌅 Golden Hour Alert! Beautiful sunset predicted"
│  └─ User: Takes commute home 30 mins later, captures golden hour shots
└─ Evening: Opens app, marks lunch challenge as complete, uploads photo

TUESDAY (Office Day)
├─ 11:30 AM: Email - "Quick Challenge: Architectural Details"
└─ User: Skips (too busy with meetings)

WEDNESDAY (Work From Home)
├─ 9:00 AM: Email - "WFH Flexible Challenge: Everyday Beauty at Home"
├─ 9:30 AM: Email - "🌫️ Special Alert: Rare fog! Perfect for moody shots"
│  └─ User: Excited about fog, grabs Ricoh for quick neighborhood walk
├─ 3:00 PM: Email reminder - "Take 20 mins for your photo challenge?"
│  └─ User: Already did morning walk, marks as complete
└─ Evening: Reviews photos, submits favorite with notes

THURSDAY (Office Day)
├─ 11:30 AM: Email - "Lunch Challenge: People Watching"
│  └─ User: Rainy, skips the challenge
└─ No evening alert (bad weather)

FRIDAY (Office Day)
├─ 11:30 AM: Email - "End-of-Week Challenge: Commute Stories"
├─ 5:00 PM: Email - "Tomorrow is Saturday! Weekend challenge preview..."
│  └─ User: Reads preview, gets excited for weekend
└─ Evening: Plans Saturday shoot based on preview

SATURDAY 🌟 (Main Photography Day)
├─ 7:00 AM: Email - "Weekend Deep Dive: Urban Geometry Challenge"
│  ├─ Full itinerary for Mission District
│  ├─ PDF attachment downloaded
│  ├─ Calendar event added
│  └─ Weather check: Perfect conditions
├─ 9:00 AM: User begins shoot with X-T5
├─ 10:00 AM: Email - "Weather Update: Perfect light at 3 PM"
├─ 12:00 PM: Lunch break, reviews shots so far
├─ 3:00 PM: Returns to location for golden hour
├─ 6:00 PM: Email - "How did it go? Share your favorite shot!"
│  └─ User: Opens app, uploads 3 best shots, writes notes
└─ Evening: Edits photos, reviews AI feedback on submissions

SUNDAY
├─ 8:00 AM: Email - "Optional Lighter Challenge (No Pressure!)"
│  └─ User: Reads but decides to rest
└─ 8:00 PM: Email - "Your Week in Photography" (Weekly Report)
    ├─ Completed 5/7 challenges (71%)
    ├─ Current streak: 12 days
    ├─ Skill improvement: Low-light exposure
    ├─ Next week focus: Leading lines
    └─ PDF report attached
```

User Journey: First Week

Day 1: Signup & Onboarding

1. User signs up with email/password
2. Dashboard loads with welcome message
3. **Onboarding wizard appears:**
   - Set your schedule (work days, hours, lunch time)
   - Choose primary photography day (Saturday pre-selected)
   - Enable lunch break challenges? (Yes/No)
   - Enable evening alerts? (Yes/No)
   - Add your gear (Fujifilm X-T5, Ricoh GR IIIx, etc.)
   - Set photography interests (street, portrait, architecture)
   - Choose notification preferences (email primary)
4. **Integration options:**
   - “Connect Google Calendar?” (optional, to detect WFH days)
   - “Enable location services?” (optional, for location-based suggestions)
5. Profile complete → Dashboard shows:
   - “Your first challenge arrives tomorrow!”
   - Current gear list
   - Empty progress stats (0 challenges)

Day 2: First Challenge

1. **11:30 AM** - Email arrives: “Your First Foto Challenge!”
2. User clicks to open in app
3. Challenge displays:
   - Simple micro-challenge (beginner-friendly)
   - Clear instructions
   - Single achievable goal
   - “Accept Challenge” or “Skip Today” buttons
4. User accepts, goes to lunch, takes photo
5. Returns to app, clicks “Mark Complete”
6. Upload photo (optional but encouraged)
7. **AI Feedback appears:** “Great start! Your use of natural light…”
8. First stat updated: **1/1 challenges completed (100%)**

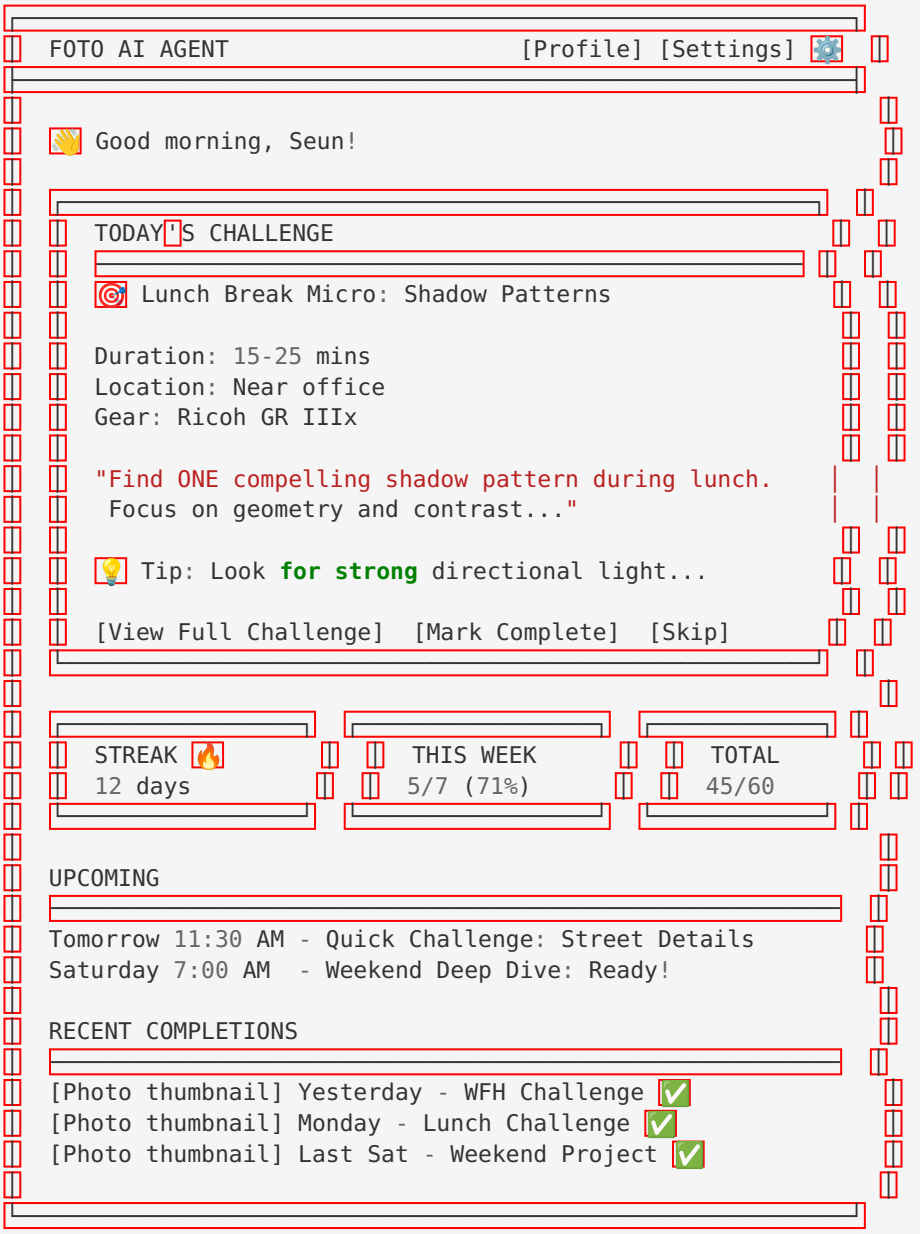
Day 7: First Weekend

1. **Saturday 7:00 AM** - Email: First weekend challenge
2. Detailed itinerary provided
3. User follows plan, shoots for 3 hours
4. Returns home, uploads 5 photos
5. Dashboard updates: “4 challenges this week (57%)”
6. **Sunday evening** - First weekly report arrives
   - Progress summary
   - Encouragement message
   - Preview of next week

---

10. Dashboard & UI Components

Main Dashboard Layout



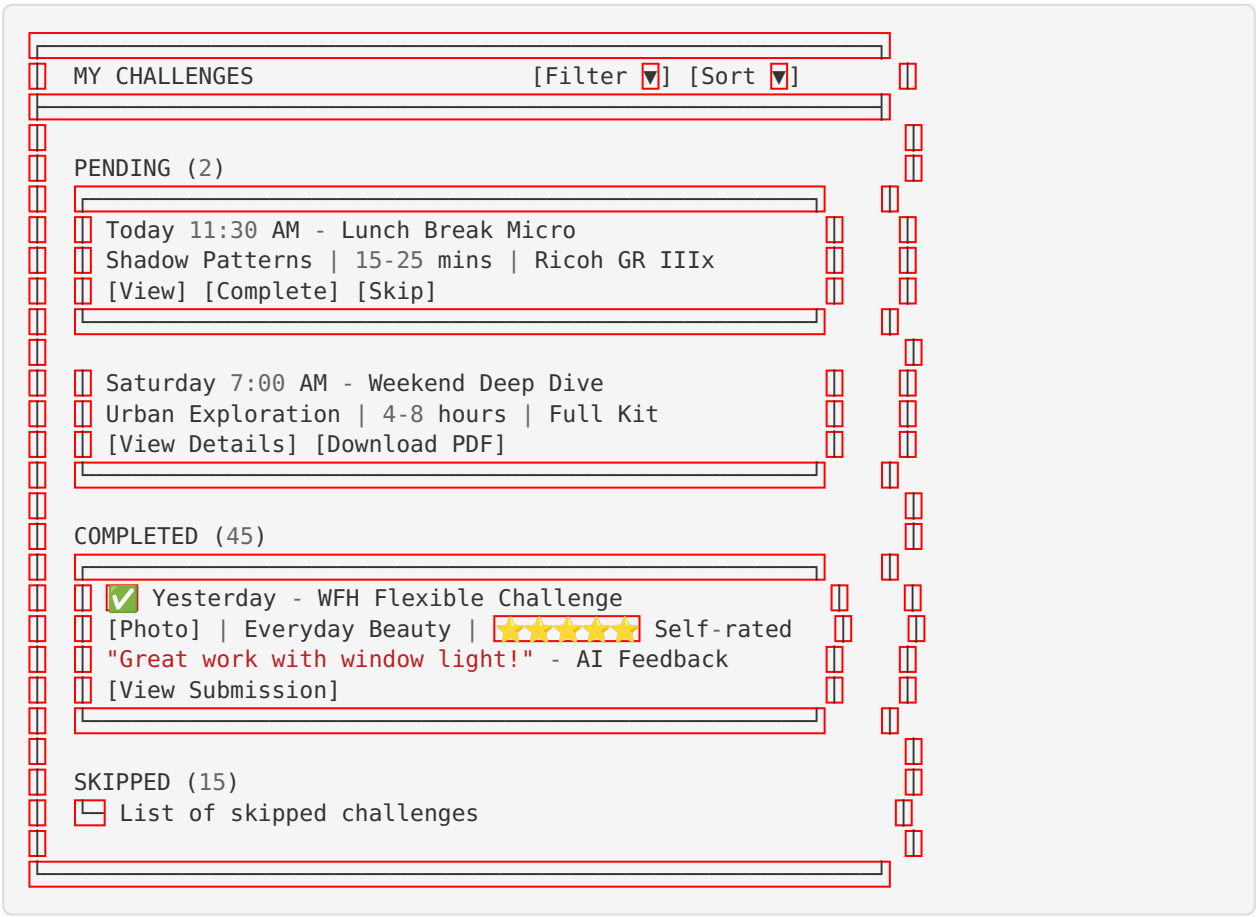
TABS: [Dashboard] [Challenges] [Progress] [Itineraries] [Settings]

Pages & Components

1. Dashboard (Home)

- Today's challenge card
- Quick stats (streak, completion rate)
- Upcoming challenges preview
- Recent submissions gallery
- Weather widget
- Quick actions (mark complete, skip, upload)

2. Challenges Page



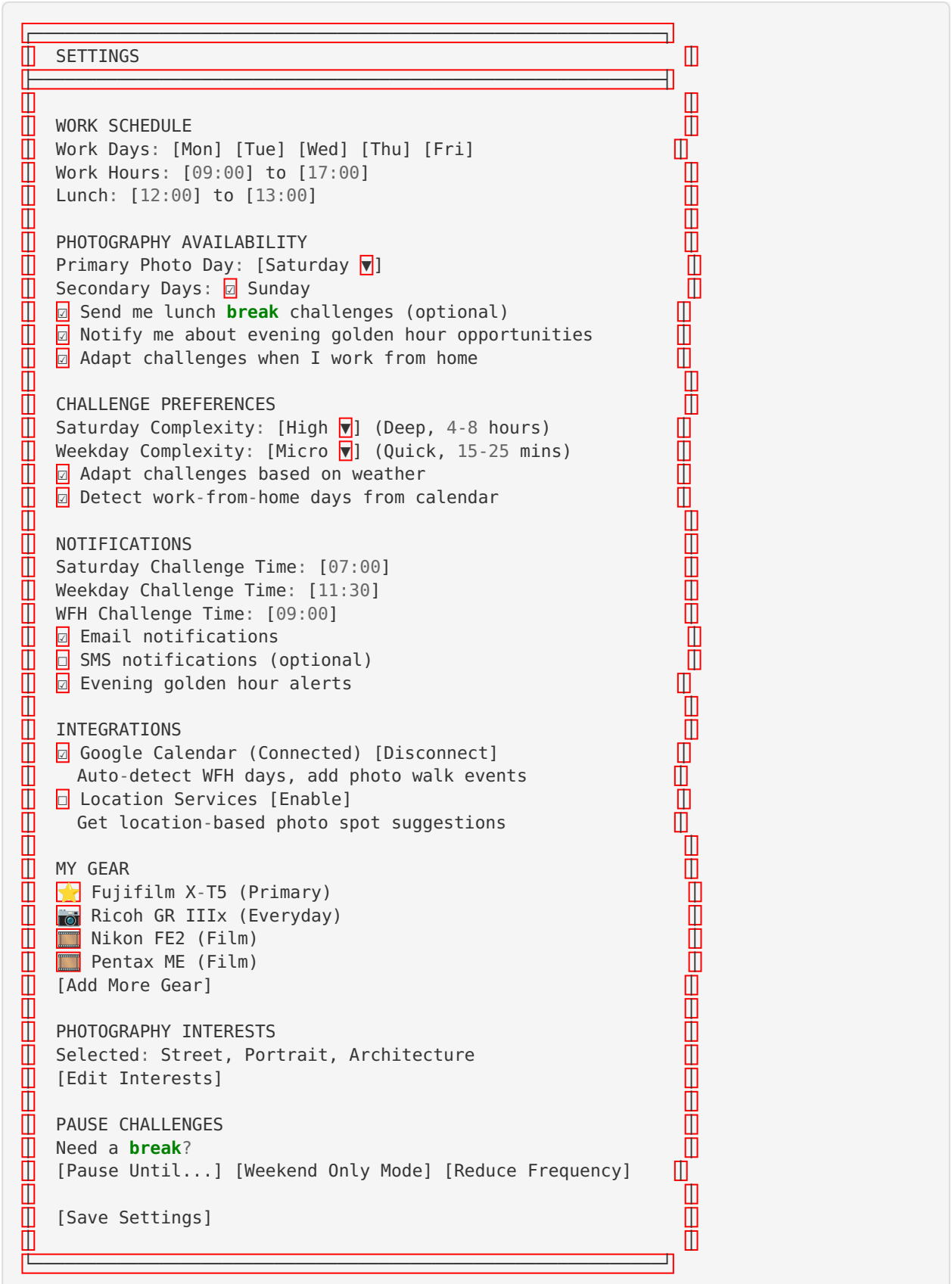
3. Progress Page



4. Itineraries Page

- List of generated itineraries
- Status: Generated, In Progress, Completed
- Download PDF option
- View on map
- Calendar integration status

5. Settings Page



11. Scheduled Tasks & Automation

Daemon/Scheduled Task Specifications

Task 1: Daily Challenge Generator

```
{
  name: "daily_challenge_generator",
  schedule: "Variable based on user preferences",

  // Multiple schedules for different challenge types
  schedules: [
    {
      type: "SATURDAY_CHALLENGE",
      cron: "0 7 * * 6", // Saturday 7:00 AM
      allUsers: false,
      filter: "users with Saturday as primary day"
    },
    {
      type: "SUNDAY_CHALLENGE",
      cron: "0 8 * * 0", // Sunday 8:00 AM
      allUsers: false,
      filter: "users who enabled Sunday challenges"
    },
    {
      type: "WEEKDAY_LUNCH_CHALLENGE",
      cron: "0 11 * * 1-5", // Weekdays 11:00 AM (sends 30 min before lunch)
      allUsers: false,
      filter: "users with lunchBreakPhotography enabled"
    },
    {
      type: "WFH_CHALLENGE",
      cron: "0 9 * * 1-5", // Weekdays 9:00 AM
      allUsers: false,
      filter: "users with WFH detected from calendar today"
    }
  ],

  process: async (userId) => {
    // 1. Determine challenge type based on context
    // 2. Generate challenge with AI
    // 3. Save to database
    // 4. Send notification
    // 5. Update progress stats
    // 6. Log action

    return await determineAndSendChallenge(userId);
  }
}
```

Task 2: Weather Monitor & Alerts

```
{
  name: "weather_monitor",
  schedule: "0 */3 * * *", // Every 3 hours

  process: async (userId) => {
    const user = await getUser(userId);
    const weather = await getWeatherForecast(user.location);

    // Check for special opportunities
    const opportunities = [];

    // Fog alert
    if (weather.condition === "Fog" || weather.condition === "Mist") {
      opportunities.push({
        type: "FOG_ALERT",
        message: "🌫️ Rare fog! Perfect for moody photography",
        urgency: "high",
        window: "next 2 hours"
      });
    }

    // Golden hour alert (check 2 hours before sunset)
    const now = new Date();
    const sunset = new Date(weather.sunset);
    const hoursUntilSunset = (sunset - now) / (1000 * 60 * 60);

    if (hoursUntilSunset > 1.5 && hoursUntilSunset < 2 &&
        weather.cloudCover < 70 &&
        user.preferences.eveningPhotography) {
      opportunities.push({
        type: "GOLDEN_HOUR_ALERT",
        message: "🌅 Beautiful sunset predicted! Golden hour at " +
                 formatTime(sunset),
        urgency: "medium",
        window: "45-90 mins"
      });
    }

    // Storm clearing
    if (weather.condition === "Thunderstorm" &&
        weather.forecast.next3Hours === "Clear") {
      opportunities.push({
        type: "DRAMATIC_SKY_ALERT",
        message: "⛈️ Storm clearing! Dramatic skies incoming",
        urgency: "high",
        window: "next 1-2 hours"
      });
    }

    // Send alerts
    for (const opp of opportunities) {
      await sendWeatherAlert(user, opp);
      await logAgentAction(userId, "WEATHER_ALERT", opp);
    }

    return opportunities.length;
  }
}
```

Task 3: Weekly Progress Report

```
{
  name: "weekly_report",
  schedule: "0 20 * * 0", // Sunday 8:00 PM

  process: async (userId) => {
    // Get user and progress
    const user = await getUser(userId);
    const progress = await getUserProgress(userId);

    // Get this week's challenges
    const weekStart = startOfWeek(new Date());
    const weekEnd = endOfWeek(new Date());

    const weekChallenges = await db.dailyChallenge.findMany({
      where: {
        userId: userId,
        issuedDate: { gte: weekStart, lte: weekEnd }
      },
      include: { submissions: true }
    });

    // Calculate week stats
    const completed = weekChallenges.filter(c => c.status === 'completed').length;
    const total = weekChallenges.length;
    const completionRate = total > 0 ? (completed / total * 100).toFixed(0) : 0;

    // Identify improvements
    const improvements = analyzeSkillImprovement(weekChallenges);

    // Generate next week focus
    const nextWeekFocus = await generateNextWeekFocus(user, progress, improvements);

    // Create report
    const report = {
      weekOf: formatDate(weekStart),
      stats: {
        completed: completed,
        total: total,
        completionRate: completionRate + '%',
        streak: progress.currentStreak,
        longestStreak: progress.longestStreak
      },
      improvements: improvements,
      nextWeekFocus: nextWeekFocus,
      highlights: weekChallenges
        .filter(c => c.status === 'completed' && c.submissions.length > 0)
        .slice(0, 3)
    };

    // Generate PDF
    const pdfUrl = await generateWeeklyReportPDF(user, report);

    // Send email
    await sendWeeklyReport(user, report, pdfUrl);

    // Log action
    await logAgentAction(userId, "WEEKLY_REPORT", report);

    return report;
  }
}
```

Task 4: Calendar Sync & WFH Detection

```
{
  name: "calendar_sync",
  schedule: "0 6 * * 1-5", // Weekdays 6:00 AM

  process: async (userId) => {
    const user = await getUser(userId);

    // Check if calendar integration enabled
    if (!user.preferences.adaptToCalendar) {
      return { wfh: false, message: "Calendar integration disabled" };
    }

    // Get today's calendar events
    const today = new Date();
    const events = await getGoogleCalendarEvents(user, today);

    // Check for WFH indicators
    const wfhKeywords = ['work from home', 'wfh', 'remote work', 'home office'];
    const hasWFH = events.some(event =>
      wfhKeywords.some(keyword =>
        event.summary.toLowerCase().includes(keyword)
      )
    );

    // Update user context
    if (hasWFH) {
      await db.userPreferences.update({
        where: { userId: userId },
        data: { currentLocation: 'home' }
      });

      // Trigger WFH challenge instead of regular lunch challenge
      await scheduleWFHChallenge(userId);

      await logAgentAction(userId, "WFH_DETECTED", {
        date: today,
        triggerSource: "calendar"
      });
    }

    return { wfh: hasWFH };
  }
}
```

Task 5: Streak Management

```
{
  name: "streak_manager",
  schedule: "0 23 * * *", // Every day at 11:00 PM

  process: async (userId) => {
    const user = await getUser(userId);
    const progress = await getUserProgress(userId);

    // Check if user completed today's challenge
    const today = new Date();
    const todayChallenge = await db.dailyChallenge.findFirst({
      where: {
        userId: userId,
        issuedDate: {
          gte: startOfDay(today),
          lte: endOfDay(today)
        }
      }
    });

    if (todayChallenge && todayChallenge.status === 'completed') {
      // Continue streak
      const newStreak = progress.currentStreak + 1;
      const newLongest = Math.max(newStreak, progress.longestStreak);

      await db.userProgress.update({
        where: { userId: userId },
        data: {
          currentStreak: newStreak,
          longestStreak: newLongest,
          lastStreakUpdate: today
        }
      });

      // Milestone notifications
      if (newStreak === 7) {
        await sendMilestoneNotification(user, "7_DAY_STREAK");
      } else if (newStreak === 30) {
        await sendMilestoneNotification(user, "30_DAY_STREAK");
      } else if (newStreak === 100) {
        await sendMilestoneNotification(user, "100_DAY_STREAK");
      }

    } else if (!todayChallenge || todayChallenge.status === 'skipped') {
      // Break streak
      if (progress.currentStreak > 0) {
        await db.userProgress.update({
          where: { userId: userId },
          data: {
            currentStreak: 0,
            lastStreakUpdate: today
          }
        });
      }
    }

    return { streak: progress.currentStreak };
  }
}
```

12. Progress Tracking & Analytics

Analytics Functions

Calculate Completion Rates

```
function calculateCompletionRate(challenges) {
  const total = challenges.length;
  const completed = challenges.filter(c => c.status === 'completed').length;

  // Overall
  const overall = total > 0 ? completed / total : 0;

  // By type
  const byType = {
    saturday: calculateTypeRate(challenges, 'WEEKEND_DEEP_DIVE'),
    lunchBreak: calculateTypeRate(challenges, 'LUNCH_MICRO'),
    afterWork: calculateTypeRate(challenges, 'AFTER_WORK'),
    wfh: calculateTypeRate(challenges, 'WORK_FROM_HOME')
  };

  // By difficulty
  const byDifficulty = {
    micro: calculateDifficultyRate(challenges, 'micro'),
    short: calculateDifficultyRate(challenges, 'short'),
    medium: calculateDifficultyRate(challenges, 'medium'),
    deep: calculateDifficultyRate(challenges, 'deep')
  };

  return { overall, byType, byDifficulty };
}

function calculateTypeRate(challenges, type) {
  const filtered = challenges.filter(c => c.challengeType === type);
  const completed = filtered.filter(c => c.status === 'completed').length;
  return filtered.length > 0 ? completed / filtered.length : 0;
}
```

Adaptive Difficulty Adjustment

```
async function adaptDifficulty(userId) {
  const recent = await db.dailyChallenge.findMany({
    where: {
      userId: userId,
      issuedDate: { gte: new Date(Date.now() - 14 * 24 * 60 * 60 * 1000) }
    }
  });

  const rates = calculateCompletionRate(recent);

  const recommendations = {};

  // Saturday challenges
  if (rates.byType.saturday < 0.3) {
    recommendations.saturday = "reduce"; // Too hard
  } else if (rates.byType.saturday > 0.9) {
    recommendations.saturday = "increase"; // Can challenge more
  } else {
    recommendations.saturday = "maintain";
  }

  // Lunch break
  if (rates.byType.lunchBreak < 0.4) {
    recommendations.lunchBreak = "simplify"; // Make easier
  } else {
    recommendations.lunchBreak = "maintain";
  }

  // Apply recommendations
  const prefs = await db.userPreferences.findUnique({
    where: { userId: userId }
  });

  if (recommendations.saturday === "reduce") {
    // Lower Saturday complexity from 'deep' to 'medium'
    if (prefs.saturdayComplexity === "high") {
      await db.userPreferences.update({
        where: { userId: userId },
        data: { saturdayComplexity: "medium" }
      });
    }
  }

  return recommendations;
}
```

Skill Progression Analysis

```
async function analyzeSkillProgression(userId) {
  // Get all completed challenges with submissions
  const completedChallenges = await db.dailyChallenge.findMany({
    where: {
      userId: userId,
      status: 'completed'
    },
    include: { submissions: true },
    orderBy: { completedDate: 'asc' }
  });

  // Group by category
  const byCategory = {};

  for (const challenge of completedChallenges) {
    const category = challenge.category;
    if (!byCategory[category]) {
      byCategory[category] = [];
    }
    byCategory[category].push(challenge);
  }

  // Analyze progression in each category
  const progression = {};

  for (const [category, challenges] of Object.entries(byCategory)) {
    const count = challenges.length;
    const avgRating = challenges
      .filter(c => c.submissions[0]?.rating)
      .reduce((sum, c) => sum + c.submissions[0].rating, 0) / count;

    // Determine level
    let level = "beginner";
    if (count > 20 && avgRating > 4) level = "advanced";
    else if (count > 10 && avgRating > 3) level = "intermediate";
    else if (count > 5) level = "developing";

    progression[category] = {
      level: level,
      challengesCompleted: count,
      averageRating: avgRating,
      trend: calculateTrend(challenges)
    };
  }

  return progression;
}

function calculateTrend(challenges) {
  // Compare first half vs second half ratings
  const mid = Math.floor(challenges.length / 2);
  const firstHalf = challenges.slice(0, mid);
  const secondHalf = challenges.slice(mid);

  const avgFirst = firstHalf
    .filter(c => c.submissions[0]?.rating)
    .reduce((sum, c) => sum + c.submissions[0].rating, 0) / firstHalf.length;

  const avgSecond = secondHalf
    .filter(c => c.submissions[0]?.rating)
    .reduce((sum, c) => sum + c.submissions[0].rating, 0) / secondHalf.length;
```

```
  if (avgSecond > avgFirst + 0.5) return "improving";
  if (avgSecond < avgFirst - 0.5) return "declining";
  return "stable";
}
```

---

13. API Endpoints

Authentication Endpoints

```
// POST /api/auth/signup
// Create new user account
{
  email: string,
  password: string,
  name: string
}

// POST /api/auth/login
// (Handled by NextAuth - /api/auth/[...nextauth])

// POST /api/auth/logout
// (Handled by NextAuth)
```

User & Preferences Endpoints

```
// GET /api/user/profile
// Get user profile and preferences

// PUT /api/user/preferences
// Update user preferences
{
  workDays: string[],
  primaryPhotoDay: string,
  lunchBreakPhotography: boolean,
  // ... all preference fields
}

// POST /api/user/gear
// Add gear to user inventory
{
  category: string,
  brand: string,
  model: string,
  isPrimary: boolean
}

// DELETE /api/user/gear/:id
// Remove gear from inventory
```

Challenge Endpoints

```
// GET /api/challenges
// Get all user challenges
// Query params: ?status=pending&limit=10

// GET /api/challenges/:id
// Get specific challenge details

// POST /api/challenges/:id/complete
// Mark challenge as complete
{
  photoUrl?: string,
  notes?: string,
  rating?: number (1-5)
}

// POST /api/challenges/:id/skip
// Skip a challenge

// POST /api/challenges/generate
// Manually trigger challenge generation (admin/testing)
{
  userId: string,
  challengeType: string
}
```

Progress Endpoints

```
// GET /api/progress
// Get user progress stats

// GET /api/progress/analytics
// Get detailed analytics
// Returns: completion rates, skill progression, streaks

// GET /api/progress/report
// Generate and download progress report PDF
```

Itinerary Endpoints

```
// GET /api/itineraries
// Get all user itineraries

// GET /api/itineraries/:id
// Get specific itinerary

// POST /api/itineraries/generate
// Generate new itinerary
{
  location: string,
  date: Date,
  duration: string
}

// GET /api/itineraries/:id/pdf
// Download itinerary as PDF
```

Integration Endpoints

```
// POST /api/integrations/calendar/connect
// Connect Google Calendar

// POST /api/integrations/calendar/disconnect
// Disconnect Google Calendar

// GET /api/integrations/calendar/events
// Get today's calendar events (check WFH)

// POST /api/integrations/weather/current
// Get current weather
{
  location: string
}

// POST /api/integrations/weather/forecast
// Get weather forecast
{
  location: string,
  days: number
}
```

Agent Endpoints (Admin/Testing)

```
// POST /api/agent/challenge/send
// Manually send challenge
{
  userId: string,
  challengeType: string
}

// POST /api/agent/weather-alert/send
// Manually send weather alert
{
  userId: string,
  alertType: string
}

// GET /api/agent/actions
// Get agent action log
// Query params: ?userId=xxx&limit=50

// POST /api/agent/report/weekly
// Manually trigger weekly report
{
  userId: string
}
```

---

14. Implementation Steps

Phase 1: Foundation (Database & Auth)

Estimated Time: 3-4 hours

1. ✅ Initialize PostgreSQL database

2. ✅ Create Prisma schema (all models)
3. ✅ Generate Prisma client and push schema
4. ✅ Set up NextAuth authentication
5. Create seed script with test user + sample data
   - Test user: sesal78@gmail.com
   - Sample gear (Fujifilm X-T5, Ricoh GR IIIx, etc.)
   - Sample preferences (Saturday primary, etc.)
6. Test authentication flow (signup, login, logout)

Phase 2: Core API Endpoints

Estimated Time: 4-5 hours

1. User & Preferences APIs
   - GET/PUT /api/user/profile
   - GET/PUT /api/user/preferences
   - POST/DELETE /api/user/gear

2. Challenge APIs
   - GET /api/challenges (with filters)
   - GET /api/challenges/:id
   - POST /api/challenges/:id/complete
   - POST /api/challenges/:id/skip

3. Progress APIs
   - GET /api/progress
   - GET /api/progress/analytics

Phase 3: AI Challenge Generation

Estimated Time: 3-4 hours

1. Create challenge generation logic (lib/challenge-generator.ts)
   - Context analyzer
   - Decision engine
   - Foto ChatLLM API integration

2. Implement challenge types
   - WEEKEND_DEEP_DIVE
   - LUNCH_MICRO
   - AFTER_WORK
   - WORK_FROM_HOME

3. Weather adaptation logic

4. Test challenge generation with different contexts

Phase 4: External Integrations

Estimated Time: 4-5 hours

1. Weather API (OpenWeatherMap)
   - Current weather
   - Forecast
   - Condition checking

2. Email Service (SendGrid)
   - Email templates (HTML)
   - Send challenge emails
   - Send alerts
   - Weekly reports

3. Google Calendar API (optional but recommended)
   - OAuth setup
   - Event fetching
   - WFH detection
   - Add events

4. PDF Generation
   - Itinerary PDFs
   - Progress report PDFs

Phase 5: Dashboard UI

Estimated Time: 6-8 hours

1. Layout & Navigation
   - Main dashboard
   - Navigation tabs
   - Responsive design

2. Dashboard Page
   - Today’s challenge card
   - Stats widgets
   - Recent completions gallery
   - Weather widget

3. Challenges Page
   - List view with filters
   - Challenge detail view
   - Complete/skip actions
   - Upload submission

4. Progress Page
   - Stats overview
   - Completion rates by type
   - Skill progression
   - Charts/graphs

5. Settings Page
   - Work schedule form
   - Availability settings
   - Notification preferences
   - Gear management
   - Integration toggles

Phase 6: Scheduled Tasks

Estimated Time: 4-5 hours

1. Daily Challenge Generator
   - Multiple schedules (Saturday, weekday, WFH)
   - Context-based triggering

2. Weather Monitor
   - Every 3 hours check
   - Opportunity detection
   - Alert sending

3. Weekly Report Generator
   - Sunday evening execution
   - Stats compilation
   - PDF generation

4. Calendar Sync
   - Morning WFH detection
   - Challenge type adjustment

5. Streak Manager
   - Nightly streak check
   - Milestone notifications

Phase 7: Progress Tracking & Analytics

Estimated Time: 3-4 hours

1. Completion rate calculation
2. Adaptive difficulty adjustment
3. Skill progression analysis
4. Trend detection
5. Dashboard data aggregation

Phase 8: Testing & Refinement

Estimated Time: 4-6 hours

1. End-to-end testing
   - Signup flow
   - Challenge generation for different days
   - Completion flow
   - Settings updates

2. Integration testing
   - Weather API
   - Email sending
   - Calendar sync
   - PDF generation

3. Scheduled task testing
   - Run tasks manually

   - Verify notifications
   - Check database updates

4. UI/UX polish
   - Loading states
   - Error handling
   - Empty states
   - Mobile responsive

5. Performance optimization

Phase 9: Deployment

Estimated Time: 1-2 hours

1. Environment variables setup
2. Build and deploy to production
3. Test live scheduled tasks
4. Monitor agent actions
5. Verify email delivery

---

15. Testing Strategy

Manual Testing Checklist

Authentication

- [ ] Signup with new user
- [ ] Login with existing user
- [ ] Logout
- [ ] Protected routes redirect to login
- [ ] Session persists after page reload

Onboarding

- [ ] Complete onboarding wizard
- [ ] Set work schedule
- [ ] Add gear
- [ ] Set preferences
- [ ] Preferences save correctly

Challenge Generation

- [ ] Saturday morning: Generate weekend deep dive
- [ ] Weekday lunch: Generate micro challenge
- [ ] WFH day: Generate flexible challenge
- [ ] Weather adaptation: Rain changes challenge
- [ ] Completion: Mark challenge complete
- [ ] Submission: Upload photo and notes
- [ ] AI Feedback: Receives feedback on submission
- [ ] Skip: Skip a challenge

Progress Tracking

- [ ] Stats update after completion
- [ ] Streak increments correctly
- [ ] Completion rates calculate accurately
- [ ] Skill progression displays
- [ ] Charts render with data

Integrations

- [ ] Weather API returns current conditions
- [ ] Weather API returns forecast
- [ ] Email sends successfully (check inbox)
- [ ] Calendar detects WFH day
- [ ] PDF generates and downloads
- [ ] Calendar event creates

Scheduled Tasks

- [ ] Daily challenge sends at scheduled time
- [ ] Weather monitor checks conditions
- [ ] Golden hour alert sends
- [ ] Weekly report generates Sunday evening
- [ ] Streak updates nightly

UI/UX

- [ ] Dashboard loads with correct data
- [ ] All pages are responsive (mobile/tablet/desktop)
- [ ] Loading states show while fetching
- [ ] Error messages display correctly
- [ ] Empty states show when no data
- [ ] Forms validate input
- [ ] Success messages confirm actions

Test Scenarios

Scenario 1: First Week User Journey

1. New user signs up
2. Completes onboarding
3. Receives first lunch challenge Tuesday 11:30 AM
4. Completes challenge, uploads photo
5. Receives AI feedback
6. Receives Saturday morning challenge at 7:00 AM
7. Completes weekend challenge
8. Receives Sunday evening weekly report

Scenario 2: Weather Adaptation

1. Saturday morning, rain forecasted
2. User receives adapted challenge with 3 options:
   - Indoor photography

   - Embrace the rain
   - Postpone to Sunday
3. User selects indoor option
4. Challenge adjusts location and gear

Scenario 3: Work From Home

1. Monday morning, user has “WFH” in calendar
2. Agent detects WFH from calendar sync (6:00 AM)
3. Instead of lunch challenge, sends WFH flexible challenge (9:00 AM)
4. Afternoon reminder sent (3:00 PM)
5. User completes challenge
6. Progress stats update WFH completion rate

Scenario 4: Golden Hour Alert

1. Thursday afternoon, clear weather
2. Agent checks weather at 4:30 PM
3. Detects perfect golden hour conditions
4. Sends alert to user: “🌅 Beautiful sunset predicted!”
5. User opens app, sees route suggestion
6. Accepts evening challenge
7. Completes and uploads photos

---

16. Deployment Considerations

Environment Variables Required

```
# Database
DATABASE_URL="postgresql://..."

# Authentication
NEXTAUTH_SECRET="..."
NEXTAUTH_URL="https://yourdomain.com"

# LLM API
ABACUSAI_API_KEY="..."

# Weather API
OPENWEATHERMAP_API_KEY="..."

# Email Service
SENDGRID_API_KEY="..."
SENDGRID_FROM_EMAIL="foto@yourdomain.com"

# SMS Service (Optional)
TWILIO_ACCOUNT_SID="..."
TWILIO_AUTH_TOKEN="..."
TWILIO_PHONE_NUMBER="..."

# Google Calendar (Optional)
GOOGLE_CLIENT_ID="..."
GOOGLE_CLIENT_SECRET="..."
GOOGLE_REDIRECT_URI="..."

# Cloud Storage
AWS_BUCKET_NAME="..."
AWS_FOLDER_PREFIX="foto/"
```

Scheduled Task Setup

The scheduled tasks need to run on the server. Use the `perform_subtask_daemon_management` tool to set up:

1. **Daily Challenge Generator** - Multiple schedules based on user types
2. **Weather Monitor** - Every 3 hours
3. **Weekly Report** - Sunday 8:00 PM
4. **Calendar Sync** - Weekday mornings 6:00 AM
5. **Streak Manager** - Daily 11:00 PM

Performance Considerations

1. **Database Indexing**
   - Index on `userId` in all models
   - Index on `issuedDate` in DailyChallenge
   - Index on `status` in DailyChallenge

2. **Caching**
   - Cache weather data (15-minute TTL)
   - Cache user preferences (session-based)

3. **Rate Limiting**
   - Weather API: Max 60 calls/min
   - Email API: Max 100/day on free tier
   - LLM API: Reasonable limits

4. **Background Jobs**
   - Use queue system for email sending
   - Async weather checks
   - Batch user processing in scheduled tasks

Monitoring

1. **Agent Action Logs**
   - Track all agent actions in database
   - Monitor success/failure rates
   - Alert on repeated failures

2. **User Metrics**
   - Daily active users
   - Challenge completion rates
   - Email open rates
   - API response times

3. **Error Tracking**
   - Failed challenge generations
   - Email delivery failures
   - API timeout errors
   - Database connection issues

---

17. Success Metrics

Key Performance Indicators

1. **User Engagement**
   - Daily active users
   - Challenge completion rate (target: >60%)
   - Average streak length (target: >7 days)
   - Weekly report open rate

2. **Challenge Quality**
   - User rating of challenges (target: >4/5)
   - Skip rate (target: <30%)
   - Feedback satisfaction

3. **Timing Accuracy**
   - Scheduled tasks execute on time
   - Notifications sent at correct times
   - Context detection accuracy (WFH, weather)

4. **System Reliability**
   - Uptime >99%

- API success rate >95%
- Email delivery rate >95%

---

18. Future Enhancements

Phase 2 Features (Post-Launch)

1. **Social Features**
   - Share submissions with community
   - Challenge friends
   - Photography buddy matching

2. **Advanced AI Feedback**
   - Detailed photo critique
   - Technique suggestions
   - Compositional analysis

3. **Learning Paths**
   - Structured courses
   - Skill trees
   - Certifications

4. **Gear Recommendations**
   - Based on shooting style
   - Budget considerations
   - Affiliate links

5. **Location Discovery**
   - Crowdsourced photo spots
   - Hidden gems database
   - Route planning

6. **Mobile App**
   - Native iOS/Android
   - Push notifications
   - Offline mode

Summary

This specification provides a complete blueprint for transforming the Foto photography coaching chatbot into a **fully autonomous, context-aware AI Agent** that:

✅ **Understands the user's real-world schedule** (work days, WFH, weekends)
✅ **Adapts challenges to available time** (15-min lunch breaks vs 4-hour Saturday sessions)
✅ **Monitors weather for opportunities** (fog alerts, golden hour notifications)
✅ **Tracks progress and skill development** (completion rates, streaks, progression)
✅ **Learns from user behavior** (adaptive difficulty, preferred styles)
✅ **Sends timely, context-appropriate notifications** (right time, right channel)
✅ **Works autonomously 24/7** (scheduled tasks, proactive coaching)

The agent respects that **Saturday is the primary photography day** while providing **achievable micro-challenges during busy weekdays**. It’s intelligent about when to push and when to back off, making it a **true AI photography coach** rather than just a reactive chatbot.

Total estimated implementation time: 32-45 hours of focused development.

Result: A production-ready AI Agent that transforms photography skill development through intelligent, personalized, and context-aware autonomous coaching.